

LAYLA: ESCRITURA CREATIVA

DOCUMENTO DE PROYECTO

LUIS DONALDO ORTIZ GARCÍA

DESARROLLO DE SISTEMAS EN RED Y DESPLIEGUE DE SOFTWARE
LICENCIATURA EN INGENIERÍA DE SOFTWARE
PERIODO FEBRERO – JUNIO 2026
UNIVERSIDAD VERACRUZANA

Contenido

Historial de versiones	3
Introducción	4
Planificación y Cronograma de Desarrollo	5
Glosario de Términos y Definiciones	6
Requisitos	9
Contexto (incluir diagrama de contexto)	9
Clases de usuario	9
Casos de uso (análisis)	11
Prototipo de UI	30
Cliente de Escritorio	30
Cliente Web	31
Cliente Android	31
Estrategia de Implementación de UI (Enfoque MVP Académico)	32
Requisitos funcionales	34
Requisitos no funcionales	36
Diseño.....	38
Diseño arquitectónico.....	38
Vista de casos de uso (Diagrama UML de casos de uso)	38
Vista lógica (Diagrama UML de clases)	39
Vista de implementación (Diagrama UML de componentes).....	39
Vista de procesos (Diagramas UML de secuencia)	39
Vista de despliegue (Diagrama UML de despliegue)	39
Control de Acceso y Seguridad (Matriz de roles)	39
Estrategia de Clientes y Especialización de Interfaces.....	40
Abordaje de Retos de Sistemas Distribuidos	42
Uso de Arquitectura Orientada a Eventos	42
Análisis y Gestión de Riesgos Técnicos	43
Modelo de datos (Diagrama de entidad-relación)	46
Descripciones de casos de uso.....	46
Construcción	46
Selección justificada de la pila tecnológica	46

Aplicación de cliente rico	48
Aplicación de servicios (API) - Servidor	48
Estándares de codificación	48
C#	48
JavaScript	48
Kotlin	48
Reportes de análisis estático de código	49
Prácticas de construcción realizadas	49
Pruebas	49
Plan de pruebas para la Aplicación de servicios (API) y clientes	49
Procedimiento de prueba (automatizado)	49
Casos de prueba (automatizados)	49
Resultados	49
Estrategia de despliegue	49
Conclusiones	50
Apéndices	50
Apéndice A: Roadmap Tecnológico y Evolución a Producto Comercial (Layla 2.0)	50

Historial de versiones

Versión	Fecha	Resumen
1.0	11 de febrero de 2026	• Creación de estructura inicial e introducción • Creación de matriz de acceso • Creación de clases de usuario • Creación de diagrama de contexto • Creación de tabla de clases de usuario y dispositivos • Creación de RF • Creación de RNF • Creación de tabla de selección de pila tecnológica • Creación de tabla de clases de usuario y dispositivos • Creación de plan de pruebas (inicial y superficial) • Creación de lista de casos de uso • Creación de diagrama de casos de uso
1.1	12 de febrero de 2026	• Actualización de sección Introducción • Actualización de tabla de clases de usuario • Actualización de RF • Creación de sección “Node.js y gRPC” • Actualización de tabla de selección de pila tecnológica • Nota sobre estrategia de despliegue agregada. • Actualización de RF • Actualización de RNF • Creación de sección “Uso de Arquitectura Orientada a Eventos • Eliminación de sección “Node.js y gRPC”) • Vista de despliegue agregada
1.2	16 de febrero de 2026	• Creación de casos de uso del módulo de gestión y seguridad (CU-05, CU-06, CU-07). • Creación de descripción para producción literaria y edición de manuscritos (CU-08). • Creación de casos de uso de Worldbuilding y gestión de entidades (CU-09, CU-10).

		• Creación de casos de uso del módulo de voz y colaboración (CU-11, CU-12). • Creación de casos de uso del módulo de lectura pública (CU-01, CU-02, CU-13).
1.3	17 de febrero de 2026	• Definición de la estrategia de implementación de UI (MVP Académico). • Análisis del flujo de navegación y propuesta de simplificación de componentes. • Creación de la matriz de riesgos técnicos y estrategias de mitigación (R-01 a R-06). • Definición de la estrategia de despliegue y arquitectura de contenedores. • Configuración de mecanismos de resiliencia y auto-recuperación (Self-Healing). • Creación del Apéndice A: Roadmap Tecnológico y Evolución a Layla 2.0.

Introducción

El presente proyecto, titulado "**Layla: Ecosistema de Escritura Creativa y Colaboración**", surge como una solución integral para autores y constructores de universos (*worldbuilders*) que requieren un entorno centralizado para la gestión narrativa.

Desde una perspectiva técnica, Layla se desarrolla como un **sistema distribuido heterogéneo** que implementa una arquitectura de microservicios para garantizar la escalabilidad y la tolerancia a fallos. La plataforma se compone de un ecosistema híbrido:

1. **Núcleo Transaccional (.NET 10):** Gestiona la identidad, seguridad y orquestación de sesiones de voz.
2. **Microservicio de Worldbuilding (Node.js):** Un servicio especializado encargado de la gestión de grafos de conocimiento (Neo4j) y documentos extensos (MongoDB).
3. **Comunicación Inter-Servicios:** Ambos backends se comunican mediante RabbitMQ.

Para la interfaz de usuario, se ha optado por una estrategia de **Tres Pantallas**:

1. **Cliente de Escritorio (WPF):** Una aplicación nativa de alto rendimiento para la redacción intensiva y gestión de manuscritos sin distracciones.
2. **Cliente Web (Razor Pages):** Un portal administrativo y de lectura pública para la gestión de usuarios y difusión de contenido.
3. **Aplicación Móvil (Android):** Una 'Companion App' para consultas rápidas y comunicación por voz en tiempo real.

Planificación y Cronograma de Desarrollo

Fase	Semana (Est.)	Hito (Milestone)	Entregable Técnico
I. Infraestructura	Semana 1	Despliegue de Contenedores	docker-compose operativo con SQL Server, Mongo y API .NET respondiendo HTTP 200 OK.
	Semana 2	Identidad y Seguridad	Sistema de Login/Registro funcional con emisión de JWT. Cliente WPF con pantalla de autenticación.
II. Núcleo (Escritura)	Semana 3	Gestión de Proyectos	CRUD de Proyectos en WPF conectado a SQL Server. Persistencia de metadatos básicos.
	Semana 4	Editor de Texto	Integración de RichTextBox en WPF. Guardado de contenido extenso en MongoDB.
	Semana 5	Modo Offline	Implementación de caché local en WPF y sincronización básica al recuperar conexión.
III. Distribuido	Semana 6	Servicio de Worldbuilding	Despliegue del contenedor Node.js y conexión exitosa con Neo4j (Creación de nodos).

	Semana 7	Sincronización Asíncrona	Implementación de RabbitMQ. Evento ProjectCreated en .NET dispara creación de nodo en Neo4j.
IV. Clientes Satélite	Semana 8	Portal Web de Lectura	Cliente Razor Pages desplegado. Visualización de lista de proyectos públicos (Feed).
	Semana 9	Cliente Móvil (Android)	App nativa Kotlin conectada a la API. Login funcional y listado de obras (Solo Lectura).
V. Tiempo Real	Semana 10	Servidor de Chat	Implementación de Hub de WebSockets en .NET. Pruebas de conexión de sockets concurrentes.
	Semana 11	Streaming de Audio	Módulo de Voz en Android. Pruebas de latencia en red local.
VI. Cierre	Semana 12	Resiliencia y Defensa	Configuración de restart policies en Docker. Pruebas de estrés y documentación final.

Glosario de Términos y Definiciones

Términos del Dominio Narrativo (Negocio)

Término	Definición
Escaleta (Beat Sheet)	Estructura esquemática de la narración que desglosa la historia en eventos clave o "golpes" (beats) antes de escribir el texto completo. En Layla, funciona como el índice dinámico del proyecto.

Manuscrito	El cuerpo textual principal de la obra (capítulos y escenas). Es el contenido no estructurado que se almacena en documentos de texto enriquecido (Rich Text).
Worldbuilding	Proceso de construcción de un mundo imaginario (geografía, historia, cultura). En el sistema, se refiere al módulo que gestiona la Wiki y el Grafo de relaciones.
Wiki (Códice)	Base de conocimientos del proyecto donde se almacenan fichas de personajes, lugares y objetos. Está vinculada directamente al grafo de relaciones.
Grafo Narrativo	Representación visual y lógica de las conexiones semánticas entre las entidades del mundo (ej. <i>"Personaje A odia a Personaje B"</i>). Se implementa técnicamente en Neo4j.
Proyecto	La unidad contenedora raíz que agrupa manuscritos, escaletas, wikis y configuraciones de permisos. Equivale a una "Novela" o "Saga".

Términos Técnicos y de Arquitectura

Término	Definición
Sincronización Híbrida	Capacidad del Cliente de Escritorio (WPF) para almacenar cambios localmente (Offline First) y transmitirlos al servidor automáticamente cuando se restablece la conexión a internet.
Consistencia Eventual	Modelo de consistencia de datos utilizado en la arquitectura distribuida. Garantiza que, tras una actualización en el Backend Core (.NET), los cambios se propagarán al servicio de Worldbuilding (Node.js) mediante colas de mensajes (RabbitMQ), aunque exista un leve retraso temporal.
Push-to-Talk (PTT)	Mecanismo de transmisión de voz donde el usuario debe mantener presionado un botón para enviar audio. Se utiliza para evitar el eco y reducir el consumo de ancho de banda en la aplicación móvil.
Cliente Rico (Rich Client)	Referencia a la aplicación de escritorio (WPF) que procesa gran parte de la lógica de presentación y renderizado en el

	hardware del usuario, a diferencia de un cliente ligero (Web).
Companion App	Rol de la aplicación móvil Android, diseñada no para la producción principal (escritura), sino como herramienta de apoyo para comunicación y consulta rápida.
Nodo (Infraestructura)	En el contexto de despliegue, se refiere a una instancia de contenedor o servidor (ej. "Nodo de RabbitMQ"). Nota: No confundir con "Nodo del Grafo Narrativo".

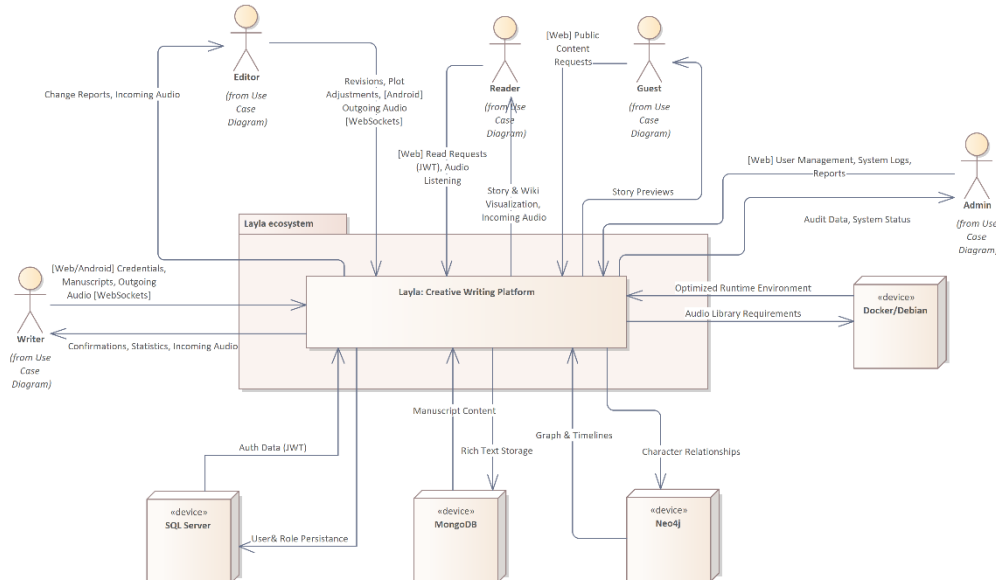
Actores y Roles

Término	Definición
Owner (Propietario)	El creador del proyecto. Tiene permisos destructivos (borrar obra) y de gestión total de accesos.
Oyente (Listener)	Estado de un usuario en el módulo de voz que puede recibir audio pero no tiene permisos para transmitir (microfono deshabilitado por el sistema).
Feed Público	Catálogo de obras marcadas con visibilidad "Pública" accesible para usuarios Invitados y Lectores, servido desde una caché de lectura optimizada.

Requisitos

Contexto (incluir diagrama de contexto)

composite structure Data Context Diagram



Clases de usuario

1. Escritor (Writer / Owner)

Es el usuario con el nivel de autoridad más alto sobre un proyecto específico.

- **Funciones principales:** Crear y eliminar proyectos, gestionar la estructura jerárquica de la obra (escaletas), redactar manuscritos con texto enriquecido y administrar los permisos de otros colaboradores.
- **Capacidad de comunicación:** Tiene acceso total al módulo de voz vía WebSockets para transmitir audio (hablar) y recibir audio (escuchar) en sesiones de co-escritura.
- **Persistencia:** Gestiona directamente registros en SQL Server (metadatos del proyecto), MongoDB (contenido) y Neo4j (grafos de personajes).

2. Editor (Collaborator)

Es un usuario invitado por el Escritor para trabajar activamente en una obra ajena.

- **Funciones principales:** Modificar el contenido de los manuscritos y actualizar las fichas de la Wiki o las relaciones en el grafo de personajes. No puede eliminar el proyecto ni cambiar los roles de otros usuarios.
- **Capacidad de comunicación:** Posee permisos de audio bidireccional (hablar y escuchar) para participar en las lluvias de ideas en tiempo real.

- **Seguridad:** Su acceso está limitado mediante el token JWT a operaciones de tipo UPDATE y POST sobre el contenido narrativo.

3. Lector Registrado (Registered Reader)

Es un usuario autenticado que consume el contenido de la plataforma.

- **Funciones principales:** Consultar manuscritos terminados, navegar por las Wikis y visualizar los mapas de relaciones de personajes.
- **Capacidad de comunicación:** Puede unirse a las sesiones de voz como oyente (solo escucha), permitiéndole seguir transmisiones de autores, pero sin capacidad de transmitir audio.
- **Restricciones:** El sistema bloquea cualquier intento de escritura o modificación de datos.

4. Invitado (Guest / Anonymous)

Usuario no autenticado que accede al sistema (similar a la navegación inicial en Pinterest).

- **Funciones principales:** Visualizar exclusivamente el contenido que los Escritores han marcado como "Público" (sinopsis, portadas o muestras gratis de capítulos).
- **Restricciones:** No posee un token JWT, por lo que el Backend rechaza automáticamente cualquier petición a los módulos de voz, edición o wikis privadas. Su objetivo es incentivar el registro en la plataforma.

Tabla de clases de usuario y dispositivos

Clase de Usuario	Dispositivo Principal	Canal Técnico	Actividad Clave
Escritor	Escritorio (WPF)	HTTPS / RabbitMQ	Producción: Redacción de manuscritos con editor de texto enriquecido y gestión offline.
Editor	Web (Razor) / WPF	HTTPS	Revisión: Corrección de estilo y sugerencias en la Wiki.

Editor (Voz)	Móvil (Android)	WebSockets	Colaboración: Participación en sesiones de lluvia de ideas por voz.
Admin	Web (Razor Pages)	HTTPS	Supervisión: Monitoreo de contenedores y usuarios.
Lector	Web / Móvil	HTTPS / WS	Consumo: Lectura de obras públicas y oyente en sesiones de voz.

Casos de uso (análisis)

ID	Nombre del Caso de Uso	Actor Principal	Módulo
CU-01	Explorar Catálogo Público (Feed)	Invitado	Acceso
CU-02	Ver Vista Previa (Sinopsis)	Invitado	Acceso
CU-03	Iniciar Sesión / Registrarse	Todos	Seguridad
CU-04	Gestionar Perfil	Usuario Registrado	Seguridad
CU-05	Crear Nuevo Proyecto	Escritor	Gestión
CU-06	Gestionar Colaboradores	Escritor	Gestión
CU-07	Configurar Privacidad	Escritor	Gestión
CU-08	Editar Manuscrito (Texto Rico)	Editor / Escritor	Escritura (MongoDB)
CU-09	Gestionar Wiki (Nodos)	Editor / Escritor	Worldbuilding (Neo4j)

CU-10	Visualizar Relaciones (Grafo)	Lector / Editor	Worldbuilding (Neo4j)
CU-11	Iniciar Sesión de Voz (Hablar)	Escritor / Editor	Voz (WebSockets)
CU-12	Unirse como Oyente	Lector	Voz (WebSockets)
CU-13	Leer Historia Completa	Lector	Lectura
CU-14	Ver Reportes del Sistema	Administrador	Admin
CU-15	Gestionar Usuarios (Ban/Roles)	Administrador	Admin

Módulo de Acceso y Seguridad

- **CU-03:** Iniciar Sesión / Registrarse
- **CU-04:** Gestionar Perfil
- **CU-15:** Gestionar Usuarios (Admin)

Módulo de Gestión y Escritura (Core)

- **CU-05:** Crear Nuevo Proyecto
- **CU-06:** Gestionar Colaboradores
- **CU-07:** Configurar Privacidad
- **CU-08:** Editar Manuscrito

Módulo de Worldbuilding (Grafos)

- **CU-09:** Gestionar Wiki
- **CU-10:** Visualizar Relaciones

Módulo de Voz y Colaboración

- **CU-11:** Iniciar Sesión de Voz (Hablar)
- **CU-12:** Unirse como Oyente

Módulo de Lectura Pública

- **CU-01:** Explorar Catálogo Público

- **CU-02:** Ver Vista Previa
- **CU-13:** Leer Historia Completa

Módulo de Acceso y Seguridad

Autenticación y Entrada

Nombre	CU-03 “Iniciar Sesión / Registrarse”
Responsable	Luis Donaldo Ortiz García
Fecha de actualización	13 de febrero de 2026
Descripción	El usuario (Invitado) ingresa sus credenciales para obtener un Token de Acceso (JWT) o crea una nueva cuenta en la plataforma para acceder a funciones restringidas.
Actor(es)	Invitado (Guest), Usuario Registrado.
Disparador	El usuario selecciona "Entrar" o "Registrarse" en la pantalla de bienvenida (Web o Desktop).
Precondiciones	PRE-1: El servicio de Backend Core (.NET) debe estar operativo. PRE-2: El usuario no debe tener una sesión activa.
Flujo Normal	1. El Sistema solicita correo electrónico y contraseña. 2. El Usuario ingresa sus credenciales. 3. El Sistema valida el formato del correo (FA-01). 4. El Usuario confirma el envío. 5. El Sistema verifica las credenciales contra SQL Server (hash de contraseña). 6. El Sistema genera un JSON Web Token (JWT) incluyendo el Rol del usuario (Escritor, Lector, etc.) y fecha de expiración. 7. El Sistema devuelve el token y redirige al Dashboard principal. 8. Termina CU.
Flujos Alternos	FA-01 Credenciales Inválidas: 1. El Sistema detecta que el correo no existe o la contraseña no coincide. 2. Muestra el mensaje: “Credenciales incorrectas”.

	- Incrementa el contador de intentos fallidos (Seguridad). - Vuelve al paso 1 del FN. FA-02 Registro de Nuevo Usuario: - El Usuario selecciona "Crear Cuenta". - Ingresa Nombre, Correo y Password. - El Sistema valida que el correo no esté duplicado en SQL Server. - El Sistema crea el registro y loguea automáticamente al usuario (va al paso 6 del FN).
Excepciones	EX-1 Base de Datos No Disponible: - El Sistema muestra “Servicio de identidad no disponible temporalmente”. - Termina CU.
Postcondiciones	POS-1: El cliente (Web/WPF/Android) almacena el JWT en almacenamiento seguro (HttpOnly Cookie o SecureStorage). POS-2: El usuario tiene acceso a los recursos permitidos por su Rol.
Reglas de negocio	RN-01: Las contraseñas deben ser almacenadas utilizando un algoritmo de hash robusto (ej. BCrypt o Argon2). RN-02: El token JWT tiene una vigencia máxima de 24 horas.
Incluye	--
Extiende	--
Calidad de servicio	- Tiempo de respuesta autenticación < 500ms. - Confidencialidad (Tráfico cifrado vía HTTPS).

Gestión de Identidad Personal

Nombre	CU-04 “Gestionar Perfil”
Responsable	Luis Donaldo Ortiz García
Fecha de actualización	13 de febrero de 2026

Descripción	El usuario autenticado actualiza su información personal, cambia su contraseña o gestiona sus preferencias de notificación.
Actor(es)	Usuario Registrado (Cualquier rol).
Disparador	El usuario selecciona "Mi Perfil" en el menú de usuario.
Precondiciones	PRE-1: El usuario tiene una sesión activa (Token JWT válido).
Flujo Normal	1. El Sistema recupera los datos del usuario desde SQL Server usando el ID del token. 2. El Sistema muestra el formulario con Nombre, Biografía y Avatar. 3. El Usuario modifica los campos deseados. 4. El Usuario presiona "Guardar Cambios". 5. El Sistema valida la entrada (longitud de textos, formato de imagen). 6. El Sistema actualiza el registro en la base de datos. 7. El Sistema confirma la operación con un mensaje de éxito "Perfil Actualizado". 8. Termina CU.
Flujos Alternos	FA-01 Cambio de Contraseña: 1. El Usuario selecciona "Seguridad". 2. Ingresa contraseña actual y nueva contraseña (x2). 3. El Sistema valida que la contraseña actual sea correcta. 4. El Sistema actualiza el hash en la BD y revoca tokens anteriores (Opcional). 5. Termina CU.
Excepciones	EX-1 Sesión Expirada: 1. El Sistema detecta que el JWT ha caducado al intentar guardar. 2. Redirige al Login (CU-03).
Postcondiciones	POS-1: Los datos actualizados son visibles inmediatamente en todos los clientes sincronizados.
Reglas de negocio	RN-03: No se permite cambiar el correo electrónico principal sin una verificación de doble factor (email de confirmación).
Incluye	--
Extiende	--

Seguridad Administrativa

Nombre	CU-15 “Gestionar Usuarios (Ban/Roles)”
Responsable	Luis Donald Ortiz García
Fecha de actualización	13 de febrero de 2026
Descripción	El Administrador visualiza la lista de usuarios registrados y ejecuta acciones de seguridad como bloqueo de cuentas (Ban) o elevación de privilegios.
Actor(es)	Administrador (Admin).
Disparador	El Admin accede al "Panel de Usuarios" en el Cliente Web.
Precondiciones	PRE-1: El usuario debe tener el rol ROLE_ADMIN en su Token JWT. PRE-2: Acceso exclusivo desde Cliente Web (Dashboard).
Flujo Normal	1. El Sistema muestra una tabla paginada de usuarios (ID, Nombre, Correo, Estado, Rol). 2. El Admin busca un usuario por correo electrónico. 3. El Admin selecciona "Editar Permisos". 4. El Sistema muestra las opciones: "Bloquear Acceso" o "Cambiar Rol". 5. El Admin cambia el estado a "Bloqueado" (Banned) por violación de términos. 6. El Admin confirma la acción con su contraseña maestra. 7. El Sistema actualiza el estado en SQL Server. 8. El Sistema fuerza el cierre de sesión del usuario afectado (Blacklist de tokens). 9. Termina CU.
Flujos Alternos	FA-01 Promover a Moderador: 1. En el paso 4, el Admin cambia el rol de "Lector" a "Editor Global" o "Admin". 2. El Sistema actualiza los permisos.
Excepciones	EX-1 Auto-Ban: 1. El Sistema impide que un Administrador se bloquee o elimine a sí mismo. 2. Muestra error: "Operación no permitida sobre su propio usuario".
Postcondiciones	POS-1: El usuario bloqueado recibe un error 403 Forbidden en su siguiente intento de acceso a la API.

Reglas de negocio	RN-04: Las acciones administrativas quedan registradas en un Log de Auditoría (Audit Trail).
Incluye	--
Extiende	--

Módulo de Gestión y Seguridad

Creación de Universos

Nombre	CU-05 “Crear Nuevo Proyecto”
Responsable	Luis Donaldo Ortiz García
Fecha de actualización	16 de febrero de 2026
Descripción	El Escritor inicializa un nuevo proyecto narrativo, estableciendo sus metadatos en SQL Server y su estructura de documentos en MongoDB.
Actor(es)	Escritor (Owner).
Disparador	El Escritor selecciona el botón flotante "+" en el Dashboard del Cliente WPF.
Precondiciones	PRE-1: El usuario debe estar autenticado. PRE-2: El servicio de Backend Core (.NET) debe estar disponible.
Flujo Normal	1. El Sistema solicita: Título, Sinopsis, Género Literario y Portada (Opcional). 2. El Escritor ingresa los datos. 3. El Sistema valida que el título no esté vacío (FA-01). 4. El Escritor confirma la creación. 5. El Sistema registra la relación User-Project en SQL Server con el rol OWNER. 6. El Sistema crea la colección de documentos vacía en MongoDB. 7. El Sistema publica el evento ProjectCreated en RabbitMQ para inicializar el grafo en Neo4j. 8. El Sistema redirige al Escritor a la vista de "Escaleta" del nuevo proyecto. 9. Termina CU.
Flujos Alternos	FA-01 Título Vacío:

	1. El Sistema resalta el campo en rojo y muestra "El título es obligatorio". 2. Vuelve al paso 2 del FN.
Excepciones	EX-1 Error de Transacción: 1. Si falla la creación en MongoDB después de insertar en SQL, el sistema hace rollback en SQL. 2. Muestra mensaje "Error de consistencia. Intente nuevamente".
Postcondiciones	POS-1: El proyecto aparece en la lista "Mis Proyectos" del usuario. POS-2: Se generan los permisos de administración (RBAC) para el creador.
Reglas de negocio	RN-01: Todo proyecto se crea por defecto como "Privado" (Borrador).
Incluye	--
Extiende	--
Calidad de servicio	· Integridad referencial garantizada.

Gestión de Equipo

Nombre	CU-06 “Gestionar Colaboradores”
Responsable	Luis Donaldo Ortiz García
Fecha de actualización	16 de febrero de 2026
Descripción	El Escritor invita a otros usuarios a participar en la obra, asignándoles roles específicos (Editor) para permitir la co-autoría o corrección.
Actor(es)	Escritor (Owner).
Disparador	El Escritor selecciona "Configuración del Equipo" en el menú del proyecto.
Precondiciones	PRE-1: El usuario debe ser el Propietario del proyecto (Role Owner).
Flujo Normal	1. El Sistema muestra la lista actual de colaboradores.

	2. El Escritor ingresa el correo electrónico del usuario a invitar. 3. El Sistema busca al usuario en la base de datos de identidad (SQL Server). 4. El Sistema valida que el usuario exista (FA-01). 5. El Escritor selecciona el rol a asignar (ej. "Editor"). 6. El Sistema registra el permiso en la tabla de Roles del proyecto. 7. El Sistema envía una notificación al usuario invitado. 8. Termina CU.
Flujos Alternos	FA-01 Usuario no encontrado: 1. El Sistema muestra "No existe un usuario con ese correo". 2. Termina el flujo de invitación.
Excepciones	--
Postcondiciones	POS-1: El usuario invitado ahora puede ver y editar el proyecto en su propio Dashboard.
Reglas de negocio	RN-02: Un proyecto no puede tener más de 5 colaboradores en la versión gratuita (Opcional). RN-03: El Owner no puede ser eliminado del proyecto.
Incluye	--
Extiende	--

Control de Visibilidad

Nombre	CU-07 “Configurar Privacidad”
Responsable	Luis Donald Ortiz García
Fecha de actualización	16 de febrero de 2026
Descripción	El Escritor cambia la visibilidad del proyecto para publicarlo en el Feed o mantenerlo oculto.
Actor(es)	Escritor (Owner).
Disparador	El Escritor cambia el switch de "Estado del Proyecto".
Precondiciones	PRE-1: El proyecto debe tener al menos un capítulo y una sinopsis.

Flujo Normal	1. El Escritor selecciona cambiar estado de "Privado" a "Público". 2. El Sistema valida que el proyecto cumpla los requisitos mínimos (RN-04). 3. El Sistema solicita confirmación: "Al hacer público el proyecto, cualquier usuario podrá leerlo". 4. El Escritor confirma. 5. El Sistema actualiza la bandera IsPublic = true en SQL Server. 6. El Sistema indexa el proyecto en el motor de búsqueda del Feed Público. 7. Termina CU.
Flujos Alternos	--
Excepciones	--
Postcondiciones	POS-1: El proyecto es visible para el actor "Invitado" y "Lector".
Reglas de negocio	RN-04: No se puede publicar un proyecto vacío (sin capítulos).
Incluye	--
Extiende	--

Producción Literaria

Nombre	CU-08 “Editar Manuscrito (Texto Rico)”
Responsable	Luis Donaldo Ortiz García
Fecha de actualización	16 de febrero de 2026
Descripción	El usuario redacta o modifica el contenido narrativo de un capítulo utilizando el editor de texto enriquecido del Cliente WPF, con soporte para guardado offline.
Actor(es)	Escritor, Editor.
Disparador	El usuario hace doble clic en un capítulo desde la "Escaleta" en el Cliente WPF.
Precondiciones	PRE-1: El usuario tiene permisos de escritura (Writer/Editor) sobre el proyecto.

	PRE-2: El capítulo no está bloqueado por otro usuario (Concurrencia Optimista).
Flujo Normal	1. El Sistema carga el contenido del capítulo desde MongoDB (JSON/HTML). 2. El Sistema renderiza el texto en el control RichTextBox de WPF. 3. El Usuario realiza modificaciones (escribe, borra, aplica negritas). 4. El Sistema guarda automáticamente los cambios en caché local cada 30 segundos (FA-01). 5. El Usuario presiona "Guardar" o sale del capítulo. 6. El Cliente WPF envía el documento actualizado a la API (.NET). 7. La API actualiza el registro en MongoDB. 8. Termina CU.
Flujos Alternos	FA-01 Modo Offline (RF-18): 1. Si el Sistema detecta pérdida de red, activa el indicador "Guardando localmente". 2. Los cambios se escriben en un archivo local cifrado. 3. Al recuperar la red, el Sistema sincroniza automáticamente el archivo local con el servidor.
Excepciones	EX-1 Conflicto de Edición: 1. Si otro usuario modificó el mismo capítulo simultáneamente, el sistema detecta discrepancia de versiones. 2. Muestra ventana de "Resolución de Conflictos" (Guardar mi versión / Descartar).
Postcondiciones	POS-1: El contenido actualizado está disponible para los Lectores. POS-2: Se genera una nueva entrada en el historial de versiones (opcional).
Reglas de negocio	RN-05: El editor debe soportar formato RTF básico (Negrita, Cursiva, Subrayado) pero no scripts ni objetos incrustados inseguros.
Incluye	--
Extiende	--
Calidad de servicio	· Experiencia de escritura fluida (sin lag de red). · Persistencia garantizada ante caídas.

Módulo de Worldbuilding

Gestión de Entidades (Wiki)

Nombre	CU-09 “Gestionar Wiki (Nodos)”
Responsable	Luis Donaldo Ortiz García
Fecha de actualización	16 de febrero de 2026
Descripción	El usuario crea o edita una ficha de entidad (Personaje, Lugar, Objeto), almacenando su contenido descriptivo y creando su representación como nodo en el grafo.
Actor(es)	Escritor, Editor.
Disparador	El usuario selecciona "Agregar Entidad" o edita una existente en el panel de Wiki.
Precondiciones	PRE-1: El usuario tiene permisos de escritura (Writer/Editor). PRE-2: El servicio de Node.js (Worldbuilding) está operativo.
Flujo Normal	1. El Sistema solicita el Tipo de Entidad (Personaje, Ubicación, Evento, Objeto). 2. El Usuario ingresa los atributos: Nombre, Alias, Descripción y Etiquetas. 3. El Usuario guarda la ficha. 4. El Sistema valida que el nombre no esté duplicado en el proyecto (FA-01). 5. El Backend Node.js crea/actualiza el nodo en Neo4j (para relaciones). 6. El Backend Node.js guarda el cuerpo de texto extenso en MongoDB. 7. El Sistema confirma la operación. 8. Termina CU.
Flujos Alternos	FA-01 Nombre Duplicado: 1. El Sistema detecta un nodo con el mismo nombre. 2. Muestra alerta: "Ya existe 'Arturo'. ¿Desea editarlo o crear un homónimo?". 3. El usuario elige la acción.
Excepciones	EX-1 Fallo Parcial (Riesgo R-01):

	1. Si Neo4j está caído, el sistema guarda en MongoDB y encola el evento en RabbitMQ para crear el nodo después. 2. Notifica al usuario: "Guardado. El grafo se actualizará en breve".
Postcondiciones	POS-1: La entidad es buscable en la Wiki y aparece visualmente en el grafo.
Reglas de negocio	RN-01: Todo nodo debe pertenecer obligatoriamente a un Proyecto (Tenant Isolation).
Incluye	--
Extiende	--

Visualización del Grafo

Nombre	CU-10 “Visualizar Relaciones (Grafo)”
Responsable	Luis Donaldto Ortiz García
Fecha de actualización	16 de febrero de 2026
Descripción	El usuario visualiza la red de conexiones entre los elementos de la historia (quién conoce a quién, dónde ocurrió qué evento) mediante una interfaz gráfica interactiva.
Actor(es)	Escritor, Editor, Lector.
Disparador	El usuario accede a la pestaña "Mapa de Relaciones" o "Grafo".
Precondiciones	PRE-1: El proyecto debe tener al menos dos entidades y una relación creada.
Flujo Normal	1. El Usuario solicita ver el grafo. 2. El Cliente (Web/WPF) solicita los datos al servicio de Node.js. 3. El Sistema recupera los nodos y aristas desde Neo4j usando el protocolo Bolt. 4. El Sistema renderiza el grafo visualmente (Nodos como círculos, Relaciones como flechas). 5. El Usuario hace clic en un nodo. 6. El Sistema despliega un panel lateral con el resumen de la ficha (traído de MongoDB).

	7. Termina CU.
Flujos Alternos	FA-01 Filtrado: 1. El Usuario selecciona "Ver solo Personajes". 2. El Sistema filtra la consulta Cypher y actualiza la vista.
Excepciones	--
Postcondiciones	POS-1: El usuario obtiene una comprensión visual de la estructura de la historia.
Reglas de negocio	RN-02 (Permisos): Los Lectores y Editores solo pueden ver; no pueden arrastrar ni modificar nodos si el grafo está en modo "Bloqueado" por el Escritor.
Incluye	--
Extiende	--

Módulo de Voz y Colaboración

Transmisión de Audio (Hablante)

Nombre	CU-11 “Iniciar Sesión de Voz (Hablar)”
Responsable	Luis Donaldo Ortiz García
Fecha de actualización	16 de febrero de 2026
Descripción	El usuario con rol de colaborador participa activamente en una sesión de voz utilizando el mecanismo Push-to-Talk para transmitir sus ideas.
Actor(es)	Escritor, Editor (Rol Emisor).
Disparador	El usuario presiona el icono de micrófono en la App Android.
Precondiciones	PRE-1: El usuario debe tener permiso de Writer o Editor en el proyecto. PRE-2: Permiso de uso de micrófono concedido en el sistema operativo (Android).
Flujo Normal	1. El Sistema conecta al usuario al Hub de WebSockets del proyecto. 2. El Sistema habilita el botón de "Push-to-Talk" (PTT).

	3. El Usuario mantiene presionado el botón PTT. 4. El Cliente Android captura el audio, lo codifica y lo envía en <i>chunks</i> binarios al servidor. 5. El Sistema muestra indicador visual "Transmitiendo" a todos los usuarios. 6. El Usuario suelta el botón. 7. El Sistema corta la transmisión. 8. Termina CU.
Flujos Alternos	FA-01 Canal Ocupado: 1. Si otro usuario está hablando, el botón PTT se deshabilita. 2. El Sistema muestra "Canal ocupado por [Nombre Usuario]".
Excepciones	EX-1 Desconexión (RNF-08): 1. Si se pierde la conexión WiFi, la app intenta reconectar 3 veces. 2. Si falla, muestra "Desconectado del chat de voz" y deshabilita el botón PTT.
Postcondiciones	POS-1: El mensaje de audio fue distribuido a todos los oyentes conectados en tiempo real.
Reglas de negocio	RN-03 (Half-Duplex): Solo un usuario puede transmitir a la vez para evitar eco y saturación.
Incluye	--
Extiende	--

Recepción de Audio (Oyente)

Nombre	CU-12 “Unirse como Oyente”
Responsable	Luis Donaldo Ortiz García
Fecha de actualización	16 de febrero de 2026
Descripción	Un usuario (Lector o Editor pasivo) ingresa a la sala de voz para escuchar la sesión de trabajo sin intervenir activamente.
Actor(es)	Lector, Editor, Escritor (Rol Receptor).
Disparador	El usuario selecciona "Escuchar Sesión" en el Cliente Web o Android.

Precondiciones	PRE-1: El proyecto tiene una sesión de voz activa.
Flujo Normal	1. El Sistema valida el token JWT del usuario. 2. El Sistema establece la conexión WebSocket en modo "Solo Escucha". 3. El Servidor agrega al usuario a la lista de distribución del Hub. 4. Cuando un "Emisor" habla (CU-11), el Sistema recibe los paquetes de audio. 5. El Cliente (Navegador/Android) reproduce el audio en secuencia. 6. El Sistema muestra visualmente quién está hablando. 7. Termina CU.
Flujos Alternos	FA-01 Silenciar (Mute): 1. El Usuario Oyente decide silenciar la salida de audio localmente. 2. El Sistema deja de reproducir sonido, pero mantiene la conexión al socket.
Excepciones	--
Postcondiciones	POS-1: El usuario aparece en la lista de "Audiencia" del proyecto.
Reglas de negocio	RN-04: Un Lector nunca puede activar el micrófono (el sistema no solicita permiso de Micrófono para este rol).
Incluye	--
Extiende	--

Módulo de Lectura Pública

Descubrimiento de Contenido

Nombre	CU-01 “Explorar Catálogo Público”
Responsable	Luis Donaldo Ortiz García
Fecha de actualización	16 de febrero de 2026
Descripción	El usuario navega por el "Feed" o catálogo de descubrimiento, visualizando las portadas y sinopsis de las obras que han sido marcadas como públicas por sus autores.

Actor(es)	Invitado (Guest), Lector.
Disparador	El usuario accede a la página de inicio (/home) o selecciona "Explorar" en el menú.
Precondiciones	PRE-1: El servicio de Backend Core (.NET) y la base de datos SQL Server deben estar operativos.
Flujo Normal	1. El Sistema consulta en SQL Server los proyectos con la bandera IsPublic = true. 2. El Sistema recupera los metadatos (Título, Autor, Género, URL de Portada). 3. El Sistema renderiza una cuadrícula (Grid) de tarjetas de proyectos. 4. El Usuario hace scroll para ver más obras. 5. El Sistema carga más elementos bajo demanda (Paginación o Infinite Scroll). 6. Termina CU.
Flujos Alternos	FA-01 Filtrado por Género: 1. El Usuario selecciona una categoría (ej. "Ciencia Ficción"). 2. El Sistema filtra la consulta SQL. 3. El Sistema actualiza la cuadrícula mostrando solo obras coincidentes.
Excepciones	EX-1 Catálogo Vacío: 1. Si no hay obras públicas, el sistema muestra un mensaje amigable: "Sé el primero en publicar una historia".
Postcondiciones	--
Reglas de negocio	RN-01: El catálogo solo muestra obras que tengan al menos un capítulo publicado y una sinopsis definida.
Incluye	--
Extiende	--

Atracción

Nombre	CU-02 “Ver Vista Previa”
Responsable	Luis Donaldo Ortiz García
Fecha de actualización	16 de febrero de 2026

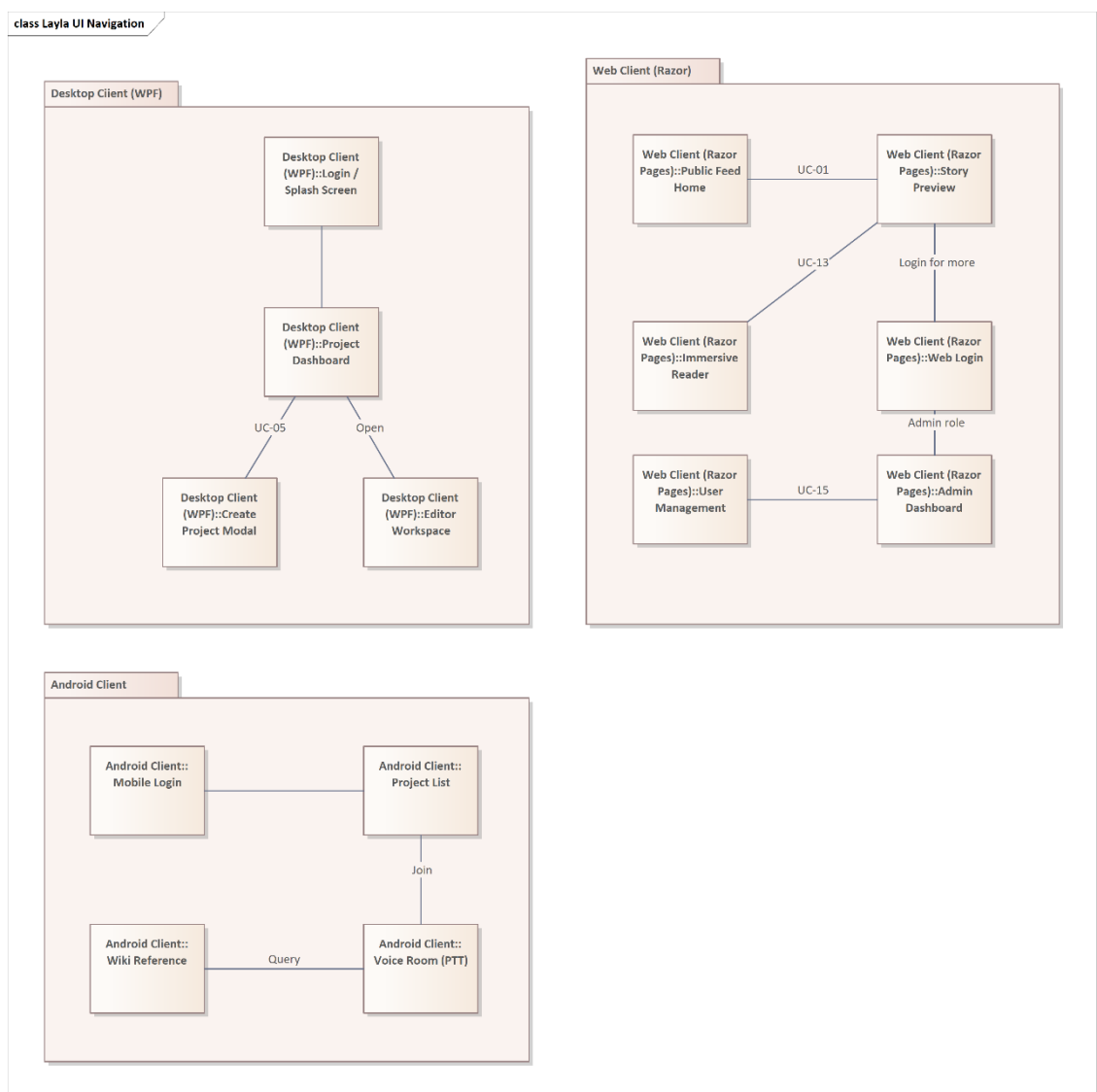
Descripción	El usuario accede a la ficha técnica de una obra para leer su sinopsis y evaluar si desea comenzar la lectura completa.
Actor(es)	Invitado (Guest).
Disparador	El usuario hace clic en una portada desde el Catálogo (CU-01).
Precondiciones	PRE-1: El proyecto seleccionado debe ser público.
Flujo Normal	1. El Sistema carga la página de detalle del proyecto. 2. El Sistema muestra: Título, Autor, Sinopsis completa, Género y Etiquetas. 3. El Sistema muestra un botón de llamada a la acción (CTA): "Leer Ahora" o "Registrarse para Leer". 4. El Usuario lee la información. 5. Termina CU.
Flujos Alternos	FA-01 Usuario ya Registrado: 1. Si el sistema detecta una sesión activa (JWT), el botón cambia a "Continuar Lectura" y lleva al CU-13.
Excepciones	--
Postcondiciones	--
Reglas de negocio	RN-02 (Acceso Limitado): Un usuario Invitado NO puede ver el contenido de los capítulos ni el grafo de relaciones, solo la metainformación.
Incluye	CU-03 "Iniciar Sesión / Registrarse" (Opcional, si el usuario decide entrar).
Extiende	--

Consumo de Contenido

Nombre	CU-13 “Leer Historia Completa”
Responsable	Luis Donald Ortiz García
Fecha de actualización	16 de febrero de 2026

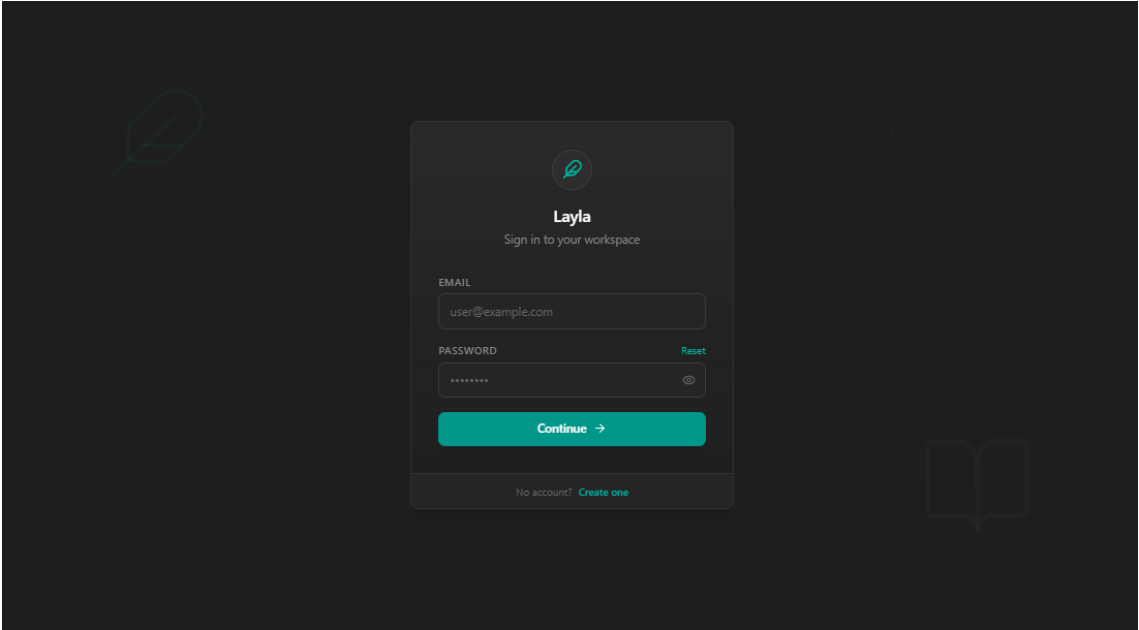
Descripción	El usuario autenticado accede a la interfaz de lectura inmersiva, visualizando el manuscrito completo y consultando la Wiki lateralmente.
Actor(es)	Lector (Registered Reader).
Disparador	El usuario selecciona "Leer" en la vista previa de un proyecto.
Precondiciones	PRE-1: El usuario debe estar autenticado (Token JWT válido). PRE-2: El proyecto debe ser público o el usuario debe ser colaborador.
Flujo Normal	1. El Sistema carga la estructura de capítulos (Índice) desde MongoDB. 2. El Sistema renderiza el texto del primer capítulo en modo "Solo Lectura" (sin barras de herramientas de edición). 3. El Usuario avanza en la lectura. 4. El Usuario hace clic en un nombre de personaje resaltado (Link inteligente). 5. El Sistema despliega un panel lateral con la ficha del personaje traída desde la Wiki (Neo4j / Mongo). 6. Termina CU.
Flujos Alternos	FA-01 Navegación por Índice: 1. El Usuario abre el menú de capítulos. 2. Selecciona "Capítulo 5". 3. El Sistema carga asíncronamente el contenido de ese documento.
Excepciones	EX-1 Contenido Bloqueado: 1. Si el autor cambia la obra a "Privada" mientras un lector lee, el sistema permite terminar el capítulo actual, pero bloquea el siguiente.
Postcondiciones	POS-1: El sistema registra el progreso de lectura (último capítulo visto) en el perfil del usuario.
Reglas de negocio	RN-03 (Integridad): El Lector tiene estrictamente prohibida la modificación del texto; la interfaz no debe cargar los scripts de edición de texto enriquecido.
Incluye	--
Extiende	CU-10 "Visualizar Relaciones" (El lector puede ver el mapa si el autor lo permite).

Prototipo de UI



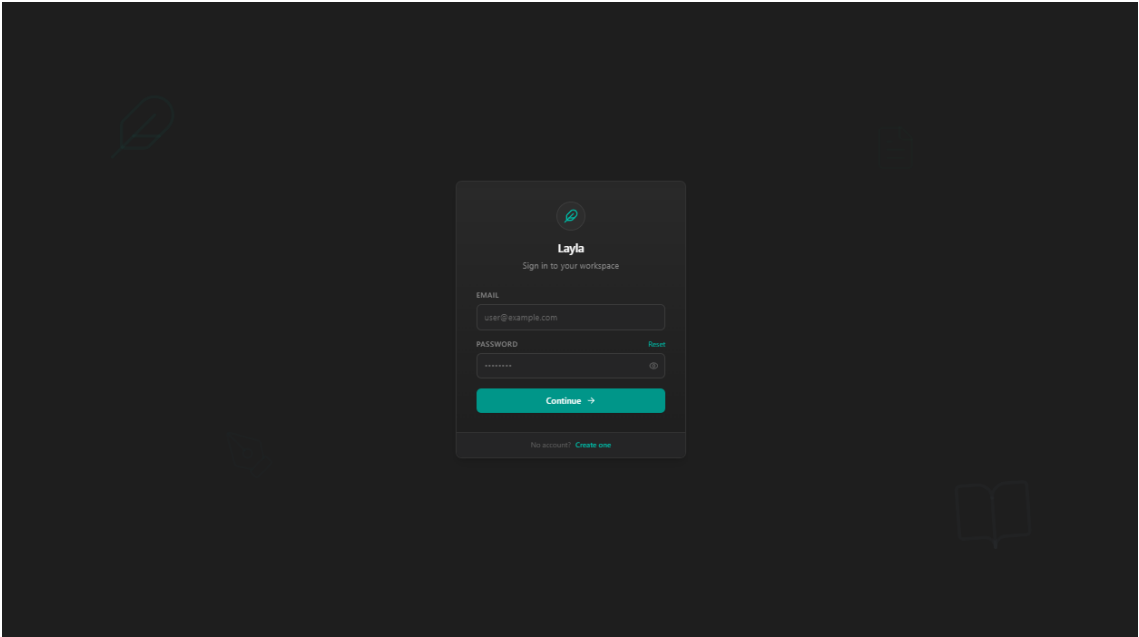
Cliente de Escritorio

Login



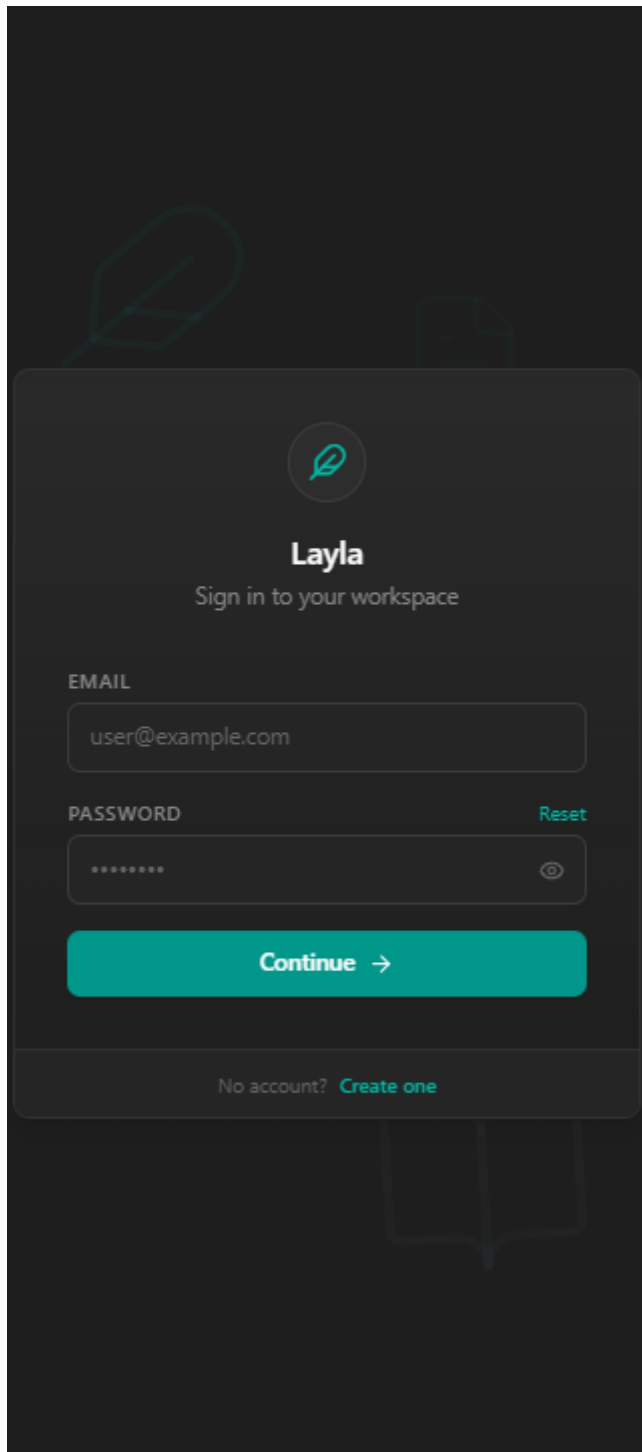
Cliente Web

Login



Cliente Android

Login



Estrategia de Implementación de UI (Enfoque MVP Académico)

Dado el alcance temporal del proyecto y los requisitos de las asignaturas ("Desarrollo de Sistemas en Red" y "Despliegue de Software"), se ha definido una estrategia de **Bajo Código y Alta Eficiencia (Low-Code / High-Efficiency)**. El objetivo es priorizar la funcionalidad distribuida y la heterogeneidad sobre la complejidad visual.

A. Análisis del Flujo de Navegación

El diseño de ventanas responde a una **arquitectura de navegación segmentada**, donde cada cliente se especializa en una tarea única para evitar la duplicidad de lógica compleja en el frontend:

1. **Cliente Escritorio (WPF):** Centraliza la carga de trabajo pesado (*Deep Work*). Su navegación se basa en pestañas (TabControl) dentro de un espacio de trabajo único, eliminando la necesidad de múltiples ventanas emergentes.
2. **Cliente Web (Razor Pages):** Centraliza la gestión y el consumo. Su flujo distingue roles post-login: los administradores son dirigidos al Dashboard y los usuarios estándar al Feed Público.
3. **Cliente Móvil (Android):** Centraliza la comunicación en tiempo real. Su flujo es lineal y estricto, optimizando el ciclo de vida de la aplicación.

B. Propuesta de Simplificación de Componentes

Para mitigar riesgos técnicos durante la fase de construcción, se utilizarán controles nativos y estandarizados en lugar de desarrollos personalizados complejos:

1. Cliente Escritorio (WPF) - Estrategia "Controles Nativos"

- **Editor de Texto:** Se utilizará el control nativo `System.Windows.Controls.RichTextBox`. El contenido se persistirá directamente como cadenas XAML/RTF o HTML en MongoDB, evitando la creación de parsers JSON personalizados.
- **Visualización de Grafos:** Se implementará mediante un Canvas estándar de WPF, representando nodos con formas básicas (Ellipse) y relaciones con líneas (Line). No se integrarán librerías de física de grafos de terceros; la visualización será estática para cumplir el requisito funcional sin aumentar la carga de procesamiento.
- **Layout:** Se priorizará el uso de Grid sobre StackPanel para garantizar la escalabilidad de los formularios de la Wiki.

2. Cliente Web (Razor Pages) - Estrategia "Server-Side Rendering"

- **Diseño Visual:** Se implementará una plantilla basada en **Bootstrap 5** para la estructura del Dashboard y el Feed, reduciendo el tiempo de maquetado CSS.
- **Lógica de Interfaz:** Se utilizará **Renderizado en Servidor (SSR)** puro. Las interacciones de navegación (ej. "Leer Capítulo") recargarán la página

completa, evitando la complejidad de gestionar estado en el cliente (SPA) con JavaScript/AJAX, lo que simplifica la depuración de errores de red.

- **Visualización de Datos:** Los reportes administrativos se presentarán mediante tarjetas de contadores simples o tablas, reservando el uso de librerías gráficas (como Chart.js) solo si el tiempo de desarrollo lo permite.

3. Cliente Móvil (Android) - Estrategia "Actividades Lineales"

- **Arquitectura UI:** Se utilizarán **Activities** nativas estándar en lugar de una arquitectura de *Single Activity* con *Fragments*, facilitando la gestión del ciclo de vida y la depuración.
- **Control PTT (Push-to-Talk):** La interfaz de voz se simplificará a un único botón (Button) con un `OnTouchListener`. La retroalimentación visual se limitará a cambios de color de estado (Rojo: Transmitiendo / Gris: Inactivo), evitando animaciones de ondas de audio complejas.

Requisitos funcionales

Módulo de Gestión de Usuarios y Seguridad (Backend .NET + SQL Server)

- **RF-01: Autenticación y Autorización.** El sistema debe permitir el registro e inicio de sesión de usuarios, generando un Token (JWT) que identifique su rol (Escritor, Editor, Lector, Admin) y permisos de sesión.
- **RF-02: Gestión de Perfiles.** El sistema debe permitir a los usuarios actualizar su información básica y gestionar sus credenciales de acceso almacenadas en SQL Server.
- **RF-03: Control de Acceso Basado en Roles (RBAC).** El sistema debe restringir el acceso a los módulos de edición y voz según la relación del usuario con el proyecto (ej. un Lector no puede editar; un Editor no puede borrar la obra).

Módulo de Escritura y Proyectos (Backend Node.js + MongoDB)

- **RF-04: Gestión de Proyectos Narrativos.** El sistema debe permitir al Escritor crear, editar y eliminar proyectos, definiendo metadatos como título, sinopsis y género.
- **RF-05: Edición de Texto (Cliente WPF).** El cliente de Escritorio debe implementar un editor de texto enriquecido (Rich Text) basado en **WPF**, capaz de manejar documentos extensos con formato (negritas, cursivas) y renderizado acelerado por hardware, garantizando una experiencia de escritura fluida superior a la de un navegador web.

- **RF-06: Persistencia Documental.** El sistema debe almacenar el contenido de los manuscritos y escaletas en una base de datos documental (MongoDB) para soportar estructuras flexibles y de gran volumen.
- **RF-07: Consistencia de Datos y Dispositivos.** La aplicación Android funcionará en modo "Companion" (Solo Lectura para manuscritos), garantizando que la fuente de la verdad para la edición de texto sea exclusivamente el Cliente Web. Esto elimina conflictos de concurrencia en la edición simultánea (Race Conditions).

Módulo de Worldbuilding y Relaciones (Backend Node.js + Neo4j)

- **RF-08: Gestión de la Wiki del Mundo.** El sistema debe permitir la creación de fichas de personajes, lugares y objetos, almacenando sus atributos en nodos de una base de datos orientada a grafos (Neo4j).
- **RF-09: Vinculación de Entidades.** El sistema debe permitir establecer y visualizar relaciones semánticas entre nodos (ej. "Personaje A es *hermano* de Personaje B") para generar el grafo de la historia.

Módulo de Colaboración y Voz (Android + WebSockets)

- **RF-10: Comunicación por Voz (Push-to-Talk).** El sistema implementará un modelo de comunicación por voz tipo *Walkie-Talkie* o *Broadcast* para optimizar el ancho de banda y garantizar la entrega de paquetes UDP/WebSockets. Esto mitiga problemas de eco y latencia en redes inestables.
- **RF-11: Roles de Voz.** El sistema distinguirá entre roles de "Emisor" (Escritor/Editor activo) y "Oyente" (Lectores/Invitados), gestionando los canales de audio mediante ASP.NET Core SignalR.
- **RF-12: Notificaciones de Presencia.** El sistema debe indicar visualmente qué usuarios están conectados y editando o hablando dentro de un proyecto activo.

Módulo de Lectura y Acceso Público (Web Razor Pages)

- **RF-13: Visualización de Contenido Público.** El portal Web debe permitir a usuarios invitados (Guest) visualizar la sinopsis y fragmentos de obras marcadas como públicas sin necesidad de autenticación.
- **RF-14: Lectura Inmersiva.** El sistema debe presentar al Lector registrado una interfaz limpia de lectura del manuscrito completo, con acceso lateral a la Wiki del mundo.

Módulo de Administración y Reportes (Web Razor Pages)

- **RF-15: Monitoreo del Sistema.** El sistema debe proveer al Administrador un tablero con el estado de los contenedores Docker y la salud de los servicios.
- **RF-16: Generación de Reportes.** El sistema debe generar gráficas estadísticas sobre el uso de la plataforma (número de proyectos, usuarios activos, volumen de datos) para la toma de decisiones.
- **RF-17: Catálogo de Obras Públicas (Feed).** El sistema debe presentar una vista de "Descubrimiento" en el portal web (Razor Pages) que liste las portadas y sinopsis de los proyectos marcados como públicos. Esta lista debe permitir el filtrado por género (ej. Fantasía, Sci-Fi) y ordenamiento por "Más Recientes".
- **RF-18: Sincronización Híbrida.** El cliente WPF debe ser capaz de operar con caché local, permitiendo al aescritor continuar redactando en caso de micro-cortes de red y sincronizando los cambios con el Backend (.NET) automáticamente al restablecerse la conexión.
 - **Criterio de Aceptación:** Para resolver conflictos de edición simultánea o desfasada (offline), el sistema implementará la política de "**Última Escritura Gana**" (**Last Write Wins - LWW**). Se utilizará la marca de tiempo del servidor (Server-Side Timestamp) al momento de recibir la petición para determinar qué versión del manuscrito prevalece, descartando versiones anteriores que lleguen con retraso.
- **RF-19: Procesamiento Asíncrono de Datos.** Las operaciones pesadas de escritura en grafos y documentos extensos deben gestionarse mediante colas de mensajes, asegurando que la interfaz de usuario no se congele esperando la confirmación de la base de datos secundaria.

Requisitos no funcionales

Operabilidad y Despliegue

- **RNF-01: Contenerización Total.** Todos los componentes del servidor (Backend .NET, Backend Node.js, SQL Server, MongoDB, Neo4j) deben estar encapsulados en contenedores **Docker**. El despliegue completo de la infraestructura debe ser ejecutable mediante un único comando de orquestación (ej. docker-compose up).
- **RNF-02: Versionado de Imágenes (Rollback Capability).** El sistema de despliegue debe utilizar etiquetado semántico de imágenes (ej. layla-api:v1.0, layla-api:v1.1). El sistema debe permitir volver a una versión anterior estable (rollback) en menos de 5 minutos simplemente cambiando la etiqueta de la imagen en el archivo de orquestación.

- **RNF-03: Separación de Configuración.** Las configuraciones sensibles (cadenas de conexión, claves JWT, puertos) no deben estar "hardcodeadas" en el código ni en la imagen del contenedor, sino que deben inyectarse mediante **Variables de Entorno** (.env) en tiempo de ejecución. Esto permite actualizar configuraciones sin recompilar ni redistribuir el código.
- **RNF-04: Persistencia de Datos Externa (Volúmenes).** Los datos de las bases de datos (SQL, Mongo, Neo4j) deben almacenarse en **Volúmenes de Docker** (Docker Volumes) independientes de los contenedores de aplicación. Esto garantiza que una actualización o destrucción del contenedor de la base de datos no resulte en pérdida de información.

Mantenibilidad y Eficiencia

- **RNF-05: Tiempo de Recuperación (RTO).** En caso de fallo crítico tras una actualización, el tiempo para restaurar el servicio a la versión anterior (Rollback) no debe exceder los **5 minutos**.
- **RNF-06: Automatización de Build.** El proceso de compilación y creación de imágenes Docker debe estar automatizado mediante scripts o un pipeline de CI (Integración Continua), asegurando que el artefacto desplegable sea idéntico en desarrollo y producción.
- **RNF-07: Hot-Swapping (Deseable).** El sistema debe permitir la actualización de los servicios de backend con un tiempo de inactividad (downtime) mínimo, preferiblemente menor a 1 minuto, mediante el reinicio rápido de contenedores ligeros (**Alpine/Debian Slim**).

Fiabilidad y Resiliencia (Reliability)

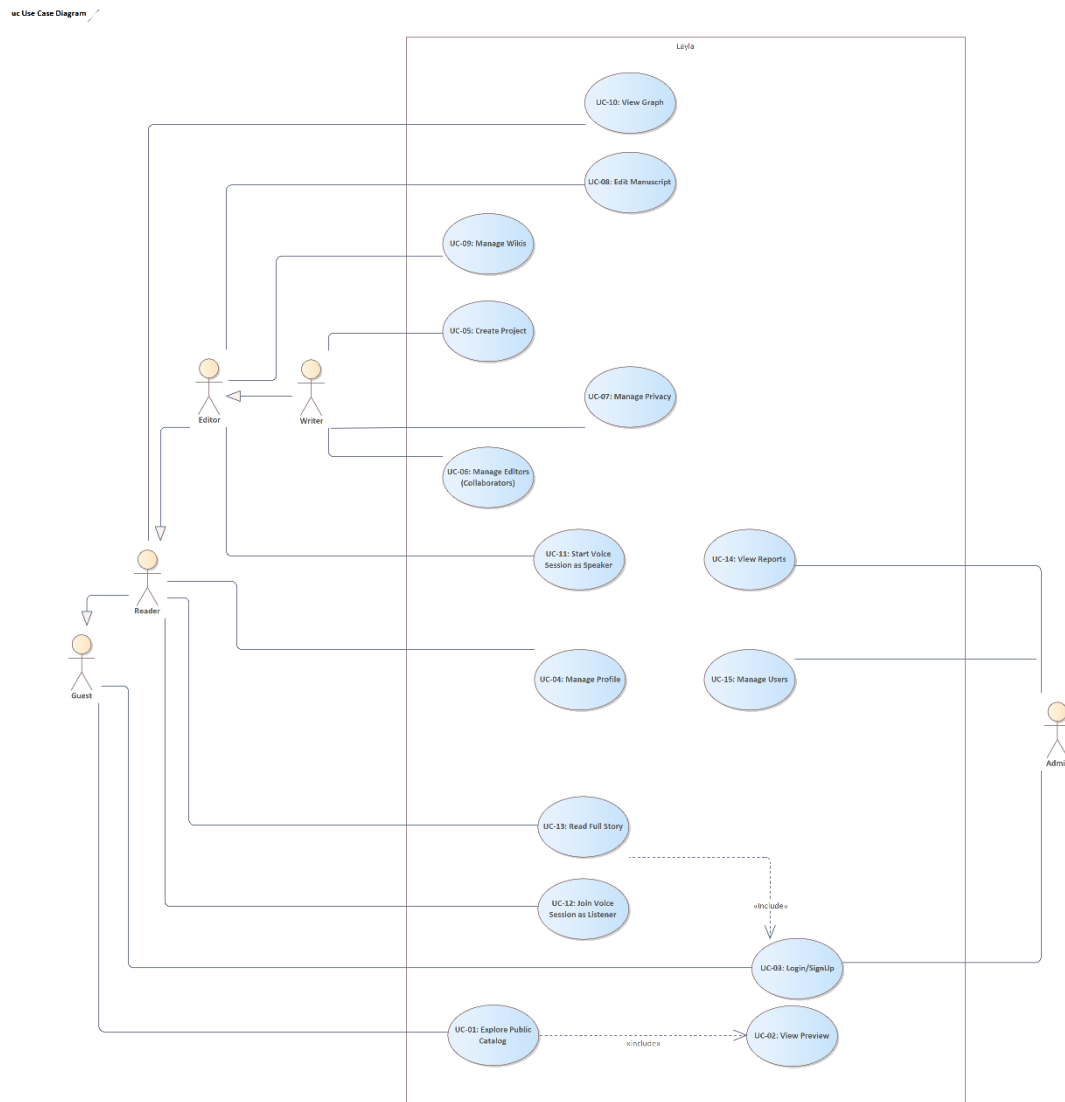
- **RNF-08: Resiliencia ante Cortes de Red (Cliente Móvil).** En caso de pérdida de conectividad durante una sesión de voz o edición, el cliente Android debe intentar la **reconexión automática** (Exponential Backoff) al socket sin cerrar la sesión del usuario. Si la reconexión falla tras 3 intentos, el sistema debe notificar al usuario y guardar el estado localmente para evitar pérdida de datos.
- **RNF-09: Recuperación Automática de Servicios (Self-Healing).** Si uno de los contenedores del backend (.NET o Node.js) falla o se cierra inesperadamente por una excepción no controlada, el orquestador (Docker) debe reiniciar el contenedor automáticamente mediante la política de reinicio (restart: on-failure) en menos de 10 segundos.
- **RNF-10: Patrón Circuit Breaker (Base de Datos).** El sistema debe implementar el patrón "Cortocircuito" (Circuit Breaker). Si la conexión con MongoDB o SQL Server falla repetidamente (timeout), el backend debe dejar

de enviar peticiones inmediatamente y devolver un error controlado (HTTP 503 Service Unavailable) en lugar de saturar la red o bloquear los hilos del servidor esperando respuesta.

Diseño

Diseño arquitectónico

Vista de casos de uso (Diagrama UML de casos de uso)

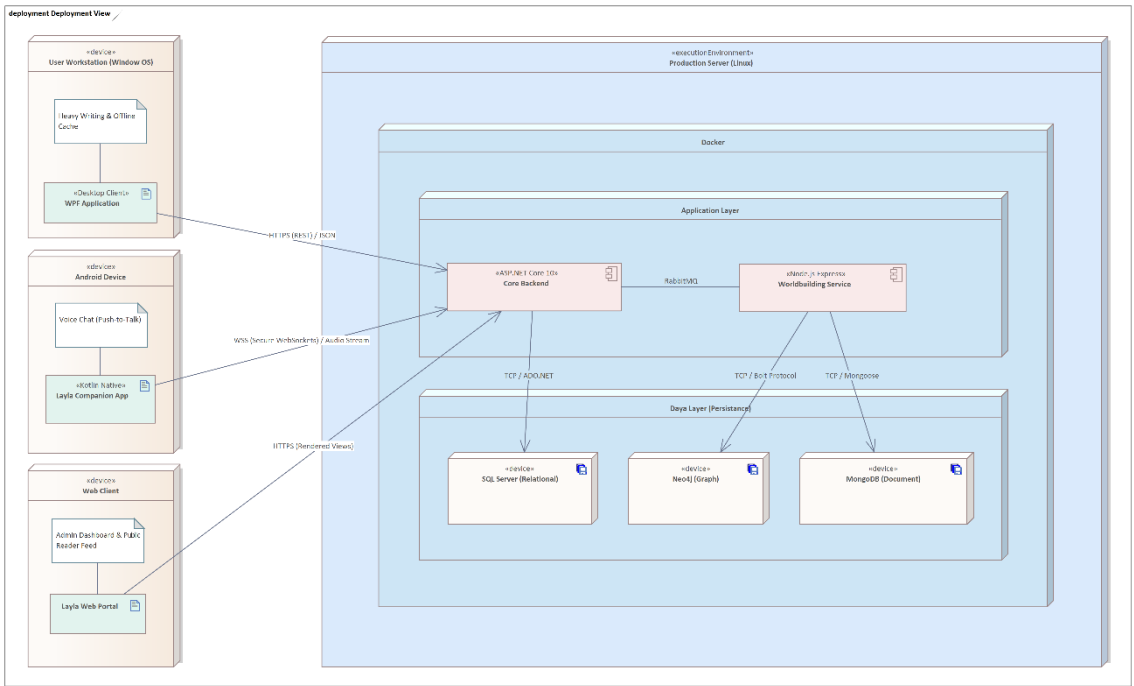


Vista lógica (Diagrama UML de clases)

Vista de implementación (Diagrama UML de componentes)

Vista de procesos (Diagramas UML de secuencia)

Vista de despliegue (Diagrama UML de despliegue)



Control de Acceso y Seguridad (Matriz de roles)

Para garantizar la integridad, confidencialidad y disponibilidad de la información en el ecosistema **Layla**, se ha implementado un modelo de **Control de Acceso Basado en Roles (RBAC)**. Este esquema permite restringir las operaciones en el sistema distribuido basándose en la identidad y el contexto del usuario respecto a cada proyecto narrativo.

Justificación Técnica: El sistema utiliza un mecanismo de autenticación mediante **JSON Web Tokens (JWT)** gestionado por el Backend de .NET 10 y validado por **Spring Security**. La autorización no es estática; el sistema evalúa dinámicamente el rol del usuario (Escritor, Editor o Lector) según el ID del proyecto solicitado. Esto resuelve el reto de la **seguridad en sistemas distribuidos**, asegurando que las peticiones a los servicios de MongoDB (manuscritos) y Neo4j (grafos) solo se ejecuten si el token contiene los privilegios adecuados.

Módulo Recurso	/ Acción	Escritor	Editor	Lector (Reg.)	Invitado	Admin

Cuentas y Auth	Registro e Inicio de Sesión	✓	✓	✓	✗	✓
Manuscritos (MongoDB)	Crear o eliminar proyectos	✓	✗	✗	✗	✓
	Modificar texto enriquecido	✓	✓	✗	✗	✓
Wiki y Tramas (Neo4j)	Gestionar nodos y relaciones	✓	✓	✗	✗	✓
	Visualizar mapas narrativos	✓	✓	✓	✗	✓
Módulo de Voz (WS)	Transmisión de audio	✓	✓	✗	✗	✓
	Recepción de audio	✓	✓	✓	✗	✓
Consultas Generales	Lectura de obras públicas	✓	✓	✓	✓	✓

Estrategia de Clientes y Especialización de Interfaces

Para abordar el reto de la **Heterogeneidad** en sistemas distribuidos, el proyecto Layla no busca replicar idénticas funcionalidades en todas las plataformas, sino que implementa una estrategia de **Especialización de Clientes**.

El sistema utiliza una API unificada (.NET 10) que sirve a tres interfaces distintas, optimizando cada una según las capacidades del dispositivo:

Cliente de Escritorio (WPF / .NET 10): "La Herramienta de Producción"

- **Objetivo:** Proveer un entorno de "Deep Work" (trabajo profundo) para el escritor.
- **Justificación Técnica:** WPF permite acceso directo al sistema de archivos para caché local y un manejo superior de eventos de teclado y renderizado de texto en comparación con el DOM de un navegador.

Cliente Web (Razor Pages): "La Herramienta de Gestión"

- **Objetivo:** Administración, análisis de datos y acceso público.

- **Justificación Técnica:** Permite el acceso universal para Lectores y Administradores sin necesidad de instalaciones, ideal para dashboards y catálogos de lectura.

Cliente Móvil (Android / Kotlin): "La Herramienta de Comunicación"

- **Objetivo:** Colaboración en tiempo real.
- **Justificación Técnica:** Aprovecha el hardware nativo (micrófono/sensores) para el módulo de voz Push-to-Talk y notificaciones.

Esta separación garantiza que el usuario siempre tenga la herramienta más adecuada para la tarea (movilidad vs. productividad), manteniendo la consistencia de los datos a través de la persistencia centralizada en MongoDB y Neo4j.

Distribución de Responsabilidades por Cliente

Característica	Escritorio (WPF)	Web (Razor)	Móvil (Android)
Escritura	Nativa (Full)	Solo Lectura / Edición Menor	Solo Lectura
Voz	Oyente / Hablante	No disponible	Nativa (Prioritaria)
Wiki	Edición de Grafos	Visualización Tabular	Consulta Rápida
Admin	No	Gestión Total	No

Aclaraciones importantes

Sobre contexto de usuarios: Un usuario puede ser **Escritor** de su propia obra y simultáneamente ser **Lector** en una obra ajena.

Manejo de invitados: Un **Invitado** tiene un alcance limitado a la lectura de contenido marcado como “público”.

Abordaje de Retos de Sistemas Distribuidos

El sistema Layla implementa estrategias específicas para mitigar los desafíos inherentes a la computación distribuida:

- **Heterogeneidad:** Se logra la interoperabilidad total entre los clientes (Android y Web) y los servicios de backend (ASP.NET Core y Node.js) mediante el intercambio de mensajes en formato JSON estándar y el uso de APIs RESTful agnósticas a la plataforma.
- **Concurrencia:** El sistema maneja múltiples escritores y editores trabajando simultáneamente. El backend de ASP.NET Core utiliza programación asíncrona para gestionar los hilos de WebSockets del chat de voz sin bloquear el servidor, mientras que Node.js maneja la concurrencia de I/O para la edición de wikis.

Resolución de Conflictos en Edición: Dado que la implementación de Transformación Operacional (OT) o Tipos de Datos Replicados Libres de Conflictos (CRDTs) excede el alcance temporal de este proyecto académico, Layla adopta un enfoque pragmático de **Last Write Wins (LWW)** para la persistencia de manuscritos.

- **Justificación:** Se asume que, en un entorno de escritura colaborativa controlada, los conflictos de edición en el mismo párrafo son poco frecuentes. LWW simplifica drásticamente la arquitectura de sincronización manteniendo la integridad del documento al priorizar la solicitud más reciente procesada por el servidor.
- **Manejo de Fallas:** Se han implementado mecanismos de resiliencia en los clientes, incluyendo colas de reintento automático para la reconexión de sockets y caché local temporal para prevenir la pérdida de datos de manuscritos ante intermitencias de red.

Uso de Arquitectura Orientada a Eventos

Comunicación Inter-Servicios: Arquitectura Orientada a Eventos Siguiendo las recomendaciones de diseño para sistemas distribuidos robustos, se utilizará un modelo de **Mensajería Asíncrona** utilizando **RabbitMQ**.

Flujo de Trabajo:

1. **Publicación (.NET):** Cuando ocurre una acción crítica (ej. "Crear Personaje"), el Backend Core publica un mensaje en la cola `worldbuilding_events`.

2. **Cola (RabbitMQ):** Actúa como buffer intermedio, garantizando la persistencia del mensaje incluso si el servicio consumidor no está disponible temporalmente.
3. **Consumo (Node.js):** El servicio de Worldbuilding escucha la cola, procesa el mensaje y actualiza Neo4j y MongoDB según corresponda.

Justificación: Este enfoque mejora la latencia percibida por el usuario (no hay espera de I/O bloqueante) y aumenta la resiliencia del sistema ante caídas parciales de servicios.

Análisis y Gestión de Riesgos Técnicos

El desarrollo de un sistema distribuido heterogéneo como **Layla** conlleva desafíos inherentes de consistencia, disponibilidad y complejidad. A continuación, se detallan los riesgos identificados y las estrategias de arquitectura diseñadas para mitigarlos.

Matriz de Riesgos y Mitigación

ID	Riesgo Identificado	Impacto	Probabilidad	Estrategia de Mitigación (Arquitectura)
R-01	Inconsistencia de Datos (Data Drift). Un fallo en el servicio de Node.js podría provocar que un personaje borrado en SQL Server siga existiendo en Neo4j (Grafo).	Alto	Media	Consistencia Eventual & Soft Deletes. Se utiliza RabbitMQ para garantizar que el mensaje de borrado llegue eventualmente. Además, se implementa "Borrado Lógico" (IsDeleted=true) en lugar de físico, facilitando la recuperación y sincronización.
R-02	Latencia en Voz (Jitter). La red puede saturarse, causando que el audio llegue	Medio	Alta	Modelo Push-to-Talk. Se elimina el eco y se reduce el ancho de banda

	entrecortado o con mucho retraso, afectando la experiencia de colaboración.			requerido al permitir solo un canal de audio activo a la vez. El protocolo WebSockets sobre TCP garantiza el orden de llegada de los paquetes.
R-03	Agotamiento de Recursos en Desarrollo. Ejecutar 6 contenedores (SQL, Mongo, Neo4j, RabbitMQ, Backends) puede saturar la memoria RAM del entorno local.	Alto	Alta	Límites de Contenedores. Configuración estricta de mem_limit en Docker Compose (ej. SQL Server limitado a 1GB) y uso de imágenes base Alpine o Slim para reducir la huella de memoria.
R-04	Fallo de Orquestación (SPOF). Si el contenedor del Backend Core (.NET) falla, todo el sistema de autenticación y acceso se detiene.	Crítico	Baja	Políticas de Auto-Recuperación. Implementación de restart: unless-stopped en Docker para el reinicio automático del servicio y Health Checks que previenen el enrutamiento de tráfico a contenedores no saludables.
R-05	Dependencia de Plataforma (WPF). El cliente de escritorio queda atado exclusivamente al	Bajo	N/A (Aceptado)	Diseño de API Abierta. Se mitiga mediante una API RESTful estandarizada que

	sistema operativo Windows.			permite, en el futuro, el desarrollo de clientes para otras plataformas (Mac/Linux) sin modificar el Backend.
R-06	Sobrescritura de Datos (Race Condition). Si dos editores modifican el mismo párrafo simultáneamente, los cambios de uno podrían perderse al aplicar Last Write Wins.	Medio	Baja	Bloqueo Optimista y LWW. El sistema favorece la simplicidad del modelo LWW. Para mitigar pérdidas críticas, el cliente WPF guarda un historial de versiones local (Local History) que permite al usuario recuperar texto sobrescrito manualmente si detecta una inconsistencia.

Detalle de Estrategias Críticas

A. Estrategia ante la Inconsistencia (R-01) Dado que no es viable implementar transacciones distribuidas (Two-Phase Commit) por su impacto en el rendimiento, el sistema adopta un modelo de **Consistencia Eventual**.

- **Acción:** Cuando el Escritor realiza un cambio crítico (ej. Renombrar Proyecto), el Backend .NET actualiza su base de datos y publica un evento en **RabbitMQ**.
- **Garantía:** Si el servicio de grafos (Node.js) está caído, el mensaje persiste en la cola. Al recuperarse el servicio, procesará los mensajes pendientes en orden, sincronizando el estado del sistema automáticamente.

B. Estrategia de Seguridad Distribuida Para evitar la latencia de red en la validación de sesiones entre microservicios:

- **Token Autosuficiente:** Se utilizan **JWT (JSON Web Tokens)** firmados con una clave secreta compartida (HS256).

- **Inyección de Secretos:** La clave de firma se inyecta en ambos contenedores (.NET y Node.js) mediante variables de entorno seguras en tiempo de ejecución. Esto permite que el microservicio de Node.js valide la autenticidad de una petición sin necesidad de realizar una llamada síncrona al servicio de identidad de .NET.

Modelo de datos (Diagrama de entidad-relación)

Estrategia de Persistencia Políglota

Para cumplir con los requisitos de optimización de datos, el sistema utiliza tres motores de base de datos:

1. SQL Server: Para la integridad referencial de usuarios, roles y permisos.
2. MongoDB: Para el almacenamiento de documentos extensos (manuscritos) y estructuras variables (fichas de wiki).
3. Neo4j: Para modelar y consultar eficientemente las relaciones complejas entre personajes y líneas temporales del universo narrativo.

Descripciones de casos de uso

Construcción

Selección justificada de la pila tecnológica

Componente	Tecnología	Justificación y Rol en el Proyecto
Backend A (Core)	ASP.NET Core 8 (C#)	Lenguaje Principal. Elegido por su robustez, tipado estático y rendimiento. Gestiona Auth, SQL Server y orquestación de SignalR.
Backend B (Helper)	Node.js (JavaScript)	Segundo Lenguaje (Heterogeneidad). Seleccionado por su eficiencia en I/O asíncrono. Gestiona la lógica de Grafos (Neo4j) y Manuscritos (MongoDB).

Comunicación entre contenedores	RabbitMQ	Interoperabilidad. Bróker de mensajes de código abierto para gestionar la comunicación asíncrona. Gestiona la llamada al servicio de Worldbuilding en el backend Node.js.
Cliente Web	Razor Pages (.NET)	Cliente Rico Web. Permite el renderizado en servidor (SSR) para la gestión compleja de textos y reportes, eliminando la necesidad de instalaciones locales.
Cliente Móvil	Android (Kotlin)	Cliente Nativo. Provee acceso al hardware de audio y notificaciones, actuando como segunda pantalla (<i>Companion App</i>).
Cliente Escritorio	WPF (.NET 10)	Cliente Rico. Seleccionado por su capacidad de integración nativa con Windows, manejo avanzado de gráficos (DirectX) para el editor de texto y reutilización de librerías C# con el Backend.
Base de Datos 1	SQL Server	Relacional. Integridad referencial para usuarios, roles y permisos.
Base de Datos 2	MongoDB	Documental. Almacenamiento flexible de manuscritos y contenido de wikis.
Base de Datos 3	Neo4j	Grafos. Modelado de relaciones complejas entre personajes y líneas temporales.
Audio	SignalR (WebSockets)	Abstracción sobre WebSockets para garantizar la reconexión automática y el manejo de grupos (Hubs) de voz.
Infraestructura	Docker + Debian Slim	Contenerización. Despliegue orquestado de ambos backends y las tres bases de datos en un entorno Linux optimizado.

Aplicación de cliente rico

Aplicación de servicios (API) - Servidor

Estándares de codificación

C#

El desarrollo en C# se alinearán estrictamente con las directrices oficiales establecidas por el creador del lenguaje.

- **Estándar de Referencia:** [C# Coding Conventions \(Microsoft\)](#) y las [Framework Design Guidelines](#).
- **Justificación:** La adopción de las convenciones de Microsoft asegura que el código sea idiomático, predecible para cualquier desarrollador del ecosistema .NET y compatible con las herramientas de análisis estático de código (como Roslyn) integradas en los entornos de desarrollo modernos.

JavaScript

Para el desarrollo en JavaScript, se adoptará uno de los estándares más estrictos y ampliamente utilizados en la industria para mitigar las ambigüedades inherentes al lenguaje.

- **Estándar de Referencia:** [Airbnb JavaScript Style Guide](#).
- **Justificación:** Esta guía proporciona un enfoque razonable y exhaustivo para escribir código JavaScript moderno (ES6 y superior). Impone reglas estrictas sobre la mutabilidad, el manejo de tipos y la sintaxis, lo que reduce drásticamente la probabilidad de errores en tiempo de ejecución y facilita la configuración de *linters* (como ESLint).

Kotlin

El código escrito en Kotlin deberá seguir las pautas idiomáticas definidas por los creadores del lenguaje, priorizando la concisión y la seguridad contra nulos.

- **Estándar de Referencia:** [Kotlin Coding Conventions \(JetBrains\)](#) y, en su caso, la [Android Kotlin Style Guide \(Google\)](#).
- **Justificación:** Seguir las convenciones oficiales de JetBrains garantiza el uso correcto de las características funcionales y orientadas a objetos de Kotlin. Además, asegura la compatibilidad con el formateador de código oficial (ktlint) y maximiza la interoperabilidad fluida con bases de código existentes en Java.

Reportes de análisis estático de código

Prácticas de construcción realizadas

Pruebas

Plan de pruebas para la Aplicación de servicios (API) y clientes

Se ha diseñado un plan de pruebas para cubrir tanto la lógica de negocio como los requisitos no funcionales de concurrencia:

- **Pruebas Unitarias (.NET):** Se utiliza **xUnit** junto con **Moq** para validar los servicios de autenticación y lógica de negocio en el Backend A (ASP.NET Core).
- **Pruebas de Integración (Node.js):** Se emplea **Jest** para verificar la correcta integración de los endpoints de la Wiki con MongoDB y Neo4j.
- **Pruebas de Carga y Concurrencia:** Uso de **JMeter** para simular múltiples conexiones simultáneas al módulo de voz y verificar la estabilidad de los WebSockets bajo carga.
- **Cobertura Objetivo:** Se busca asegurar un mínimo del 70% de cobertura de código en los módulos críticos del sistema.

Procedimiento de prueba (automatizado)

Casos de prueba (automatizados)

Resultados

Estrategia de despliegue

Arquitectura de Contenedores

El despliegue del ecosistema Layla se basa en una arquitectura de microservicios containerizados utilizando **Docker**. Se ha seleccionado una imagen base de **Debian Slim** para los servicios de backend (.NET y Node.js) debido a su equilibrio entre ligereza y compatibilidad con librerías de audio (ALSA) necesarias para el módulo de voz.

Orquestación y Ciclo de Vida (Docker Compose)

La orquestación de los servicios se define mediante un archivo docker-compose.yml que gestiona la interdependencia entre los contenedores. Se han configurado políticas de reinicio automático (restart: unless-stopped) para garantizar que, ante un fallo crítico o reinicio del servidor host, los servicios de backend y bases de datos se recuperen automáticamente sin intervención manual (Self-Healing).

Mecanismos de Resiliencia y Alta Disponibilidad

Para cumplir con los requisitos no funcionales de estabilidad (RNF-08, RNF-09, RNF-10), se implementan las siguientes estrategias:

- **Verificaciones de Salud (Health Checks):** Se configuran sondas de salud en los contenedores de base de datos (SQL Server, MongoDB). Los servicios dependientes (Backends) esperan mediante la directiva depends_on: condition: service_healthy hasta que la persistencia esté totalmente operativa, evitando errores de conexión durante el arranque (Cold Start).
- **Resiliencia Transitoria (Polly en .NET):** Se utiliza la librería Polly y la resiliencia nativa de Entity Framework Core (EnableRetryOnFailure) para manejar micro-cortes de red. El sistema reintenta automáticamente las operaciones fallidas contra la base de datos antes de reportar un error al usuario.
- **Persistencia de Datos (Volúmenes):** Se utilizan volúmenes nombrados de Docker para desacoplar los datos de los contenedores de ejecución, garantizando que la información de los manuscritos y usuarios sobreviva a la destrucción o actualización de los contenedores de base de datos.

Conclusiones

Apéndices

Apéndice A: Roadmap Tecnológico y Evolución a Producto Comercial (Layla 2.0)

Este documento anexo describe la visión a largo plazo para la plataforma **Layla**, proyectando su evolución desde la actual arquitectura académica distribuida hacia un producto comercial unificado y escalable.

Justificación de la Arquitectura Actual vs. Futura

La arquitectura actual (Layla v1.0) fue diseñada para satisfacer los requisitos de **heterogeneidad, sistemas distribuidos y redes**, priorizando la implementación

de múltiples pilas tecnológicas (.NET, Node.js, Kotlin, WPF) para demostrar competencias de integración compleja.

Sin embargo, para una etapa de comercialización y mantenimiento a largo plazo (Layla v2.0), la estrategia técnica cambiará hacia la **Unificación del Código (Code Sharing)** para reducir la deuda técnica y acelerar el despliegue de nuevas funcionalidades.

Propuesta de Stack Tecnológico Unificado (Layla v2.0)

El objetivo de la siguiente iteración será migrar de un ecosistema fragmentado a un **Monorepo basado en TypeScript**, manteniendo la robustez del Backend actual, pero unificando las tres interfaces de usuario (Web, Escritorio, Móvil).

Evolución Tecnológica

Componente	Layla v1.0 (Académico/Actual)	Layla v2.0 (Visión Comercial)	Motivación del Cambio
Cliente Web	Razor Pages (.NET)	SvelteKit + TypeScript	Migrar de renderizado en servidor (SSR) a una SPA reactiva moderna que elimine la recarga de páginas y mejore la experiencia de escritura.
Cliente Escritorio	WPF (Windows)	Tauri + Svelte (TypeScript)	Reemplazar la dependencia exclusiva de Windows por una arquitectura multiplataforma (Win/Mac/Linux) ultraligera, compartiendo el 90% del código con la web.
Cliente Móvil	Android Nativo (Kotlin)	Capacitor / Ionic	Permitir el despliegue simultáneo en iOS y Android utilizando la misma base de código web, reduciendo el

			equipo de desarrollo necesario.
Protocolo de Voz	WebSockets (TCP Raw)	WebRTC	Adoptar el estándar industrial para comunicación en tiempo real (P2P), delegando la gestión de códecs, cancelación de eco y latencia al navegador/WebView.
Backend	.NET 10 + Node.js	.NET 10 (Unificado)	Mantener .NET como núcleo por su rendimiento, evaluando la migración de los servicios de Node.js a .NET 10 (Minimal APIs) para simplificar la infraestructura de despliegue.

Estrategia de Migración

La transición hacia Layla 2.0 se ejecutará en tres fases post-académicas:

1. **Fase de Unificación de UI (Frontend):** Desarrollo de una biblioteca de componentes de interfaz en **Svelte** (Botones, Editor de Texto, Paneles) que reemplace las vistas de Razor y los controles XAML de WPF.
2. **Fase de Hibridación (Desktop):** Implementación de **Tauri** para encapsular la nueva aplicación web de Svelte, permitiendo el acceso al sistema de archivos local (Local First) necesario para el trabajo offline.
3. **Fase de Estandarización de Audio:** Sustitución del módulo de WebSockets manual por una implementación de **WebRTC**, mejorando la calidad de audio y reduciendo la carga en el servidor central (Server-Load) al pasar a una topología Peer-to-Peer donde sea posible.

Conclusión del Apéndice

Mientras que Layla v1.0 demuestra la capacidad de orquestar sistemas dispares y resolver retos de bajo nivel (sockets, hilos, serialización manual), Layla v2.0 se enfocará en la **Eficiencia del Desarrollador (Developer Experience)** y la

Escalabilidad del Producto, alineándose con las tendencias modernas de desarrollo de software SaaS (Software as a Service).